AWS VPC Peering

Two VPCs, one private connection, and the evidence that it works

Field	Value
Name	Raj Baibhav
Batch	_____
Project	01 - AWS VPC Peering (build / test / explain)
Course	EWB Cloud Labs / AWS Architect Associate - Student Project 01
Region	ap-south-1 (Mumbai)
Completed	17 October 2026
Result	6 of 6 workbook tests passed, teardown verified clean
Evidence	13 CLI transcripts and 11 console screenshots, all in the public repo

I built the whole environment through the AWS CLI instead of clicking, ran every test the workbook asks for, and kept the raw terminal output behind each claim. The lab starts with a ping that fails on purpose, ends with a security group rule removed and restored to show which control actually matters, and along the way it hit two real bugs: an ICMP type/code mistake that took packet captures on both sides to find, and a stale SSH host key that only appeared when the lab was rebuilt.

1. Submission checklist

The workbook asks for seven things. Each one is answered below and points at the section, the transcript or the screenshot that proves it.

Workbook item	Where it is in this report	Proof
Cover: name, batch, project title, Region, completion date	cover page	this PDF
Architecture diagram and inventory with actual resource IDs and private IPs	section 2	evidence/00-inventory.txt
Subnet CIDRs and all three route tables, including their subnet associations	sections 3 and 5	evidence/04, screenshots cp01-*
Active peering connection and inbound rules for each security group	sections 4 and 5	evidence/08, screenshot cp03
Baseline failure, forward and reverse ping success, and App-B SSH proof	section 6	evidence/03, 05, 06, 07
Controlled ICMP failure and recovery, with a short explanation	section 7	evidence/09, evidence/10
Cleanup evidence plus a short note on what you learned	sections 9 and 10	evidence/11, evidence/13

Nothing in this report holds a private key, a password or an access token. The key pair was made on the laptop with ssh-keygen and only its public half was imported into EC2, so the private half never left the machine and is not in the repository.

2. Architecture diagram and address plan

One internet entry point (the bastion), two private workloads, and a peering connection that carries traffic only between them. VPC B has no internet gateway, so the only path to App-B is through the bastion, and the only path between the apps is their private addresses.

```
+--------------------------------+      +--------------------------------+
| VPC A             10.10.0.0/16 |      | VPC B             10.20.0.0/16 |
| internet gateway: ewb-igw-a    |      | no internet gateway            |
|                                |      |                                |
| ewb-a-public    10.10.1.0/24   |      | ewb-b-private   10.20.1.0/24   |
|   ewb-bastion      10.10.1.10  |      |   ewb-app-b        10.20.1.10  |
|   public IPv4: yes             |      |   public IPv4: none            |
|                                |      |                                |
| ewb-a-private   10.10.2.0/24   |      |                                |
|   ewb-app-a        10.10.2.10  |      |                                |
|   public IPv4: none            |      |                                |
+----------------+---------------+      +----------------+---------------+
                 |                                       |
     10.20.0.0/16 => pcx                   10.10.0.0/16 => pcx
                 |                                       |
                 +--------- peering connection ----------+

Paths under test
  laptop --TCP 22 (my public IP /32, through ewb-igw-a)--> ewb-bastion
  ewb-bastion --VPC A local route--> ewb-app-a            (works before peering)
  ewb-bastion --10.20.0.0/16 => pcx--> ewb-app-b          (ProxyJump, needs the peering route)
  ewb-app-a <--ICMP echo, 4 of 4 each way--> ewb-app-b    (private IPs only)
```

Resource inventory with real IDs

```
# EWB VPC Peering - resource inventory
# student : Raj Baibhav   batch: _____________
# region  : ap-south-1 (Mumbai)   account: 597595166970
# built   : 2026-10-17, twice - same addressing both times

Build 2 - the console screenshots and tests 02-05 (built 17:44 IST, torn down after 18:12 IST)

RESOURCE            ID                      DETAIL
ewb-bastion         i-0f9bbae63560aaff9     10.10.1.10 + public 13.126.70.86
ewb-app-a           i-08cd773d457488487     10.10.2.10, no public IP
ewb-app-b           i-097b484f23a762579     10.20.1.10, no public IP
ewb-a-b-peer        pcx-0132d27205827aad2    active, A(requester)->B(accepter)
ewb-vpc-a           vpc-0843b8bd344059014    10.10.0.0/16
ewb-vpc-b           vpc-097437c5c5b04cb2a    10.20.0.0/16
ewb-a-public        subnet-0de70c3c480ad047c 10.10.1.0/24 (public)
ewb-a-private       subnet-07f7ab785fb0c31da 10.10.2.0/24 (private)
ewb-b-private       subnet-07407c3c1c2d1763f 10.20.1.0/24 (private)
ewb-igw-a           igw-0169b40008fdaae98    attached to VPC A
ewb-rt-a-public     rtb-0e0a730af7cfbfc2a    0.0.0.0/0->igw, 10.20/16->pcx
ewb-rt-a-private    rtb-0bd505c2786b703ab    10.20.0.0/16->pcx
ewb-rt-b-private    rtb-0e1cc69e534cb466f    10.10.0.0/16->pcx (return)
ewb-sg-bastion      sg-0a6467b762464b32f     tcp22 from MY_IP/32
ewb-sg-app-a        sg-0b9037ae2a5b95caa     tcp22+icmp type 8/code 0
ewb-sg-app-b        sg-00fa26a69eb0b9db5     tcp22+icmp type 8/code 0

Build 1 - where the ICMP bug was found (built 14:44 IST, torn down 15:43 IST)

RESOURCE            ID                      DETAIL
ewb-bastion         i-0f6a52081a6f8113b      10.10.1.10 + public 13.206.123.132
ewb-app-a           i-05aa30d91ddf22920      10.10.2.10, no public IP
ewb-app-b           i-03d1512b1a3659ff1      10.20.1.10, no public IP
ewb-a-b-peer        pcx-043967053372c4309    active, A(requester)->B(accepter)
ewb-vpc-a           vpc-06df4636df68c167f    10.10.0.0/16
ewb-vpc-b           vpc-0d483d2bd26c30745    10.20.0.0/16
ewb-a-public        subnet-0f40affc406626319 10.10.1.0/24 (public)
ewb-a-private       subnet-03b5a67650f8c48c7 10.10.2.0/24 (private)
```

```
ewb-b-private       subnet-07c5f4a60deac60e1 10.20.1.0/24 (private)
ewb-igw-a           igw-057b7c2e3f47120ab    attached to VPC A
ewb-rt-a-public     rtb-02690d674bd52e807    0.0.0.0/0->igw, 10.20/16->pcx
ewb-rt-a-private    rtb-03701f904a1bb801e    10.20.0.0/16->pcx
ewb-rt-b-private    rtb-0b1d136734b970656    10.10.0.0/16->pcx (return)
ewb-sg-bastion      sg-0cc45566a5466b21c     tcp22 from MY_IP/32
ewb-sg-app-a        sg-0c05847d87a41ccf8     tcp22+icmp (first attempt: type 8/code 8)
ewb-sg-app-b        sg-07ba1d7b79b22d488     tcp22+icmp (first attempt: type 8/code 8)

# AMI both builds: ami-0c25f859a8b518798 (al2023-ami-2023.12.20261016.0-kernel-6.1-x86_64), type t3.micro
# key pair: ewb-lab (imported public key; private key never left the laptop)
# every resource carries Project=EWB-VPC-Peering, Lab=VPC-Peering, Student=Raj Baibhav
```

3. Subnet CIDRs and the three route tables

Three custom tables, each explicitly associated with its own subnet. The private tables hold only the local VPC route until peering is added - there is no default route anywhere in VPC B, and none in the private subnet of VPC A either.

```
# route tables after peering  2026-10-17T12:16:17Z
# peering: pcx-0132d27205827aad2

### ewb-rt-a-public = rtb-0e0a730af7cfbfc2a
  subnets: subnet-0de70c3c480ad047c
  DESTINATION       TARGET                    ORIGIN       STATE
  10.10.0.0/16      local                     CreateRouteTable active
  10.20.0.0/16      pcx-0132d27205827aad2    CreateRoute  active
  0.0.0.0/0         igw-0169b40008fdaae98    CreateRoute  active

### ewb-rt-a-private = rtb-0bd505c2786b703ab
  subnets: subnet-07f7ab785fb0c31da
  DESTINATION       TARGET                    ORIGIN       STATE
  10.10.0.0/16      local                     CreateRouteTable active
  10.20.0.0/16      pcx-0132d27205827aad2    CreateRoute  active

### ewb-rt-b-private = rtb-0e1cc69e534cb466f
  subnets: subnet-07407c3c1c2d1763f
  DESTINATION       TARGET                    ORIGIN       STATE
  10.10.0.0/16      pcx-0132d27205827aad2    CreateRoute  active
  10.20.0.0/16      local                     CreateRouteTable active
```

Why the public subnet also needs a peering route

ProxyJump does not tunnel a single TCP stream end to end. The bastion opens its own connection to App-B, so the route lookup happens in the bastion's subnet table. 10.20.0.0/16 is more specific than 0.0.0.0/0, so that traffic goes to the peering connection instead of the internet. Without this route, ssh ewb-app-b would time out at the bastion while App-A to App-B still pinged.

4. Peering connection and security group rules

Peering connection	Value
ID and name	pcx-0132d27205827aad2 / ewb-a-b-peer
Status	Active (requested by VPC A, accepted in the same call, same account)
Requester	vpc-0843b8bd344059014 / ewb-vpc-a, 10.10.0.0/16, ap-south-1
Accepter	vpc-097437c5c5b04cb2a / ewb-vpc-b, 10.20.0.0/16, ap-south-1
DNS resolution	left disabled on both sides - this lab routes by IP only

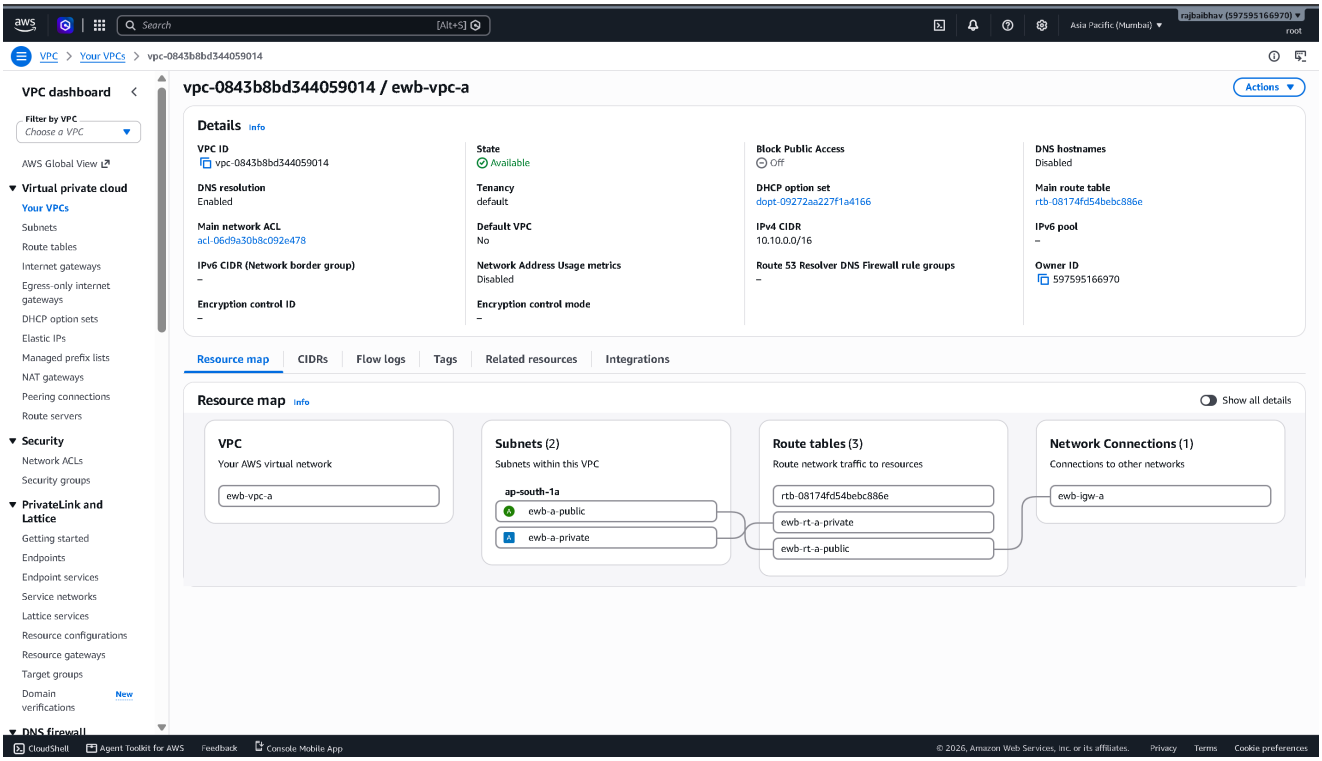
Inbound only; outbound stays allow-all and the NACLs stay default, so routing and security groups are the only variables under test. Every source is a /32 host, never 0.0.0.0/0.

Group	Inbound rule	Why that source
ewb-sg-bastion	TCP 22 from my public IPv4 /32	only my laptop touches the one public host
ewb-sg-app-a	TCP 22 from 10.10.1.10/32	administration comes from the bastion only
ewb-sg-app-a	ICMP Echo (type 8, code 0) from 10.20.1.10/32	App-B's reverse ping
ewb-sg-app-b	TCP 22 from 10.10.1.10/32	ProxyJump sends the bastion's private IP, not mine
ewb-sg-app-b	ICMP Echo (type 8, code 0) from 10.10.2.10/32	App-A's forward ping

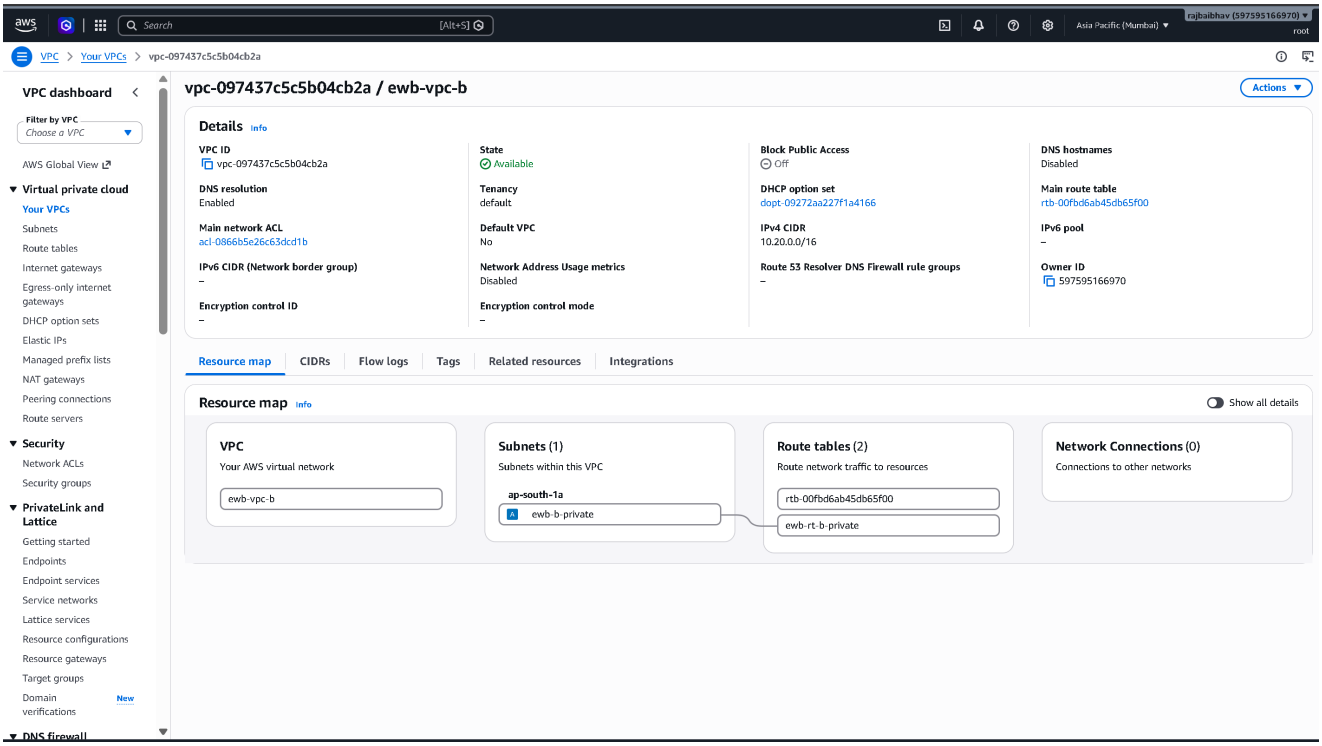
5. Console screenshots (Checkpoint 01-04)

Taken from the console while the environment was live, on 17 October 2026 between 17:58 and 18:12 IST. Each caption names the host or view, the test it supports and the result it shows.

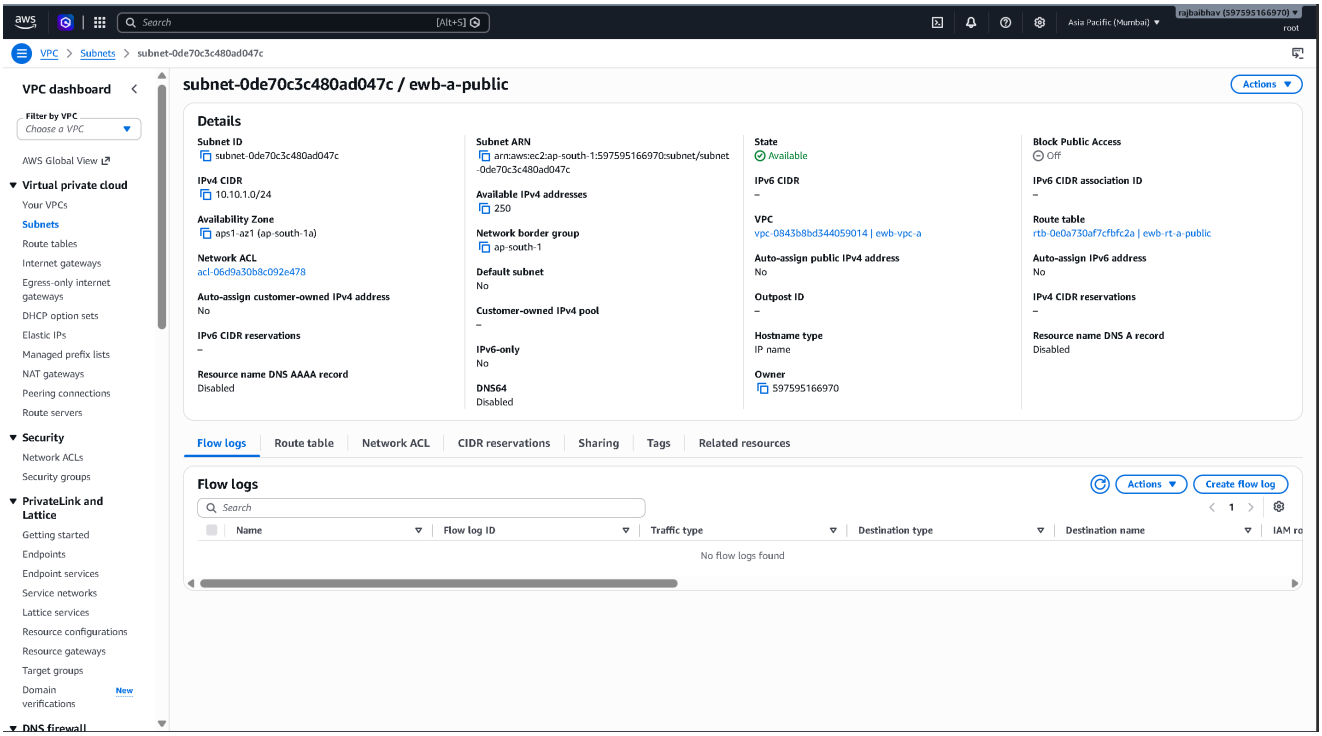
Checkpoint 01 - VPC A resource map | view: vpc-0843b8bd344059014 / ewb-vpc-a | test: subnet and route-table layout | result: 2 subnets, 3 route tables, ewb-igw-a as the only network connection.



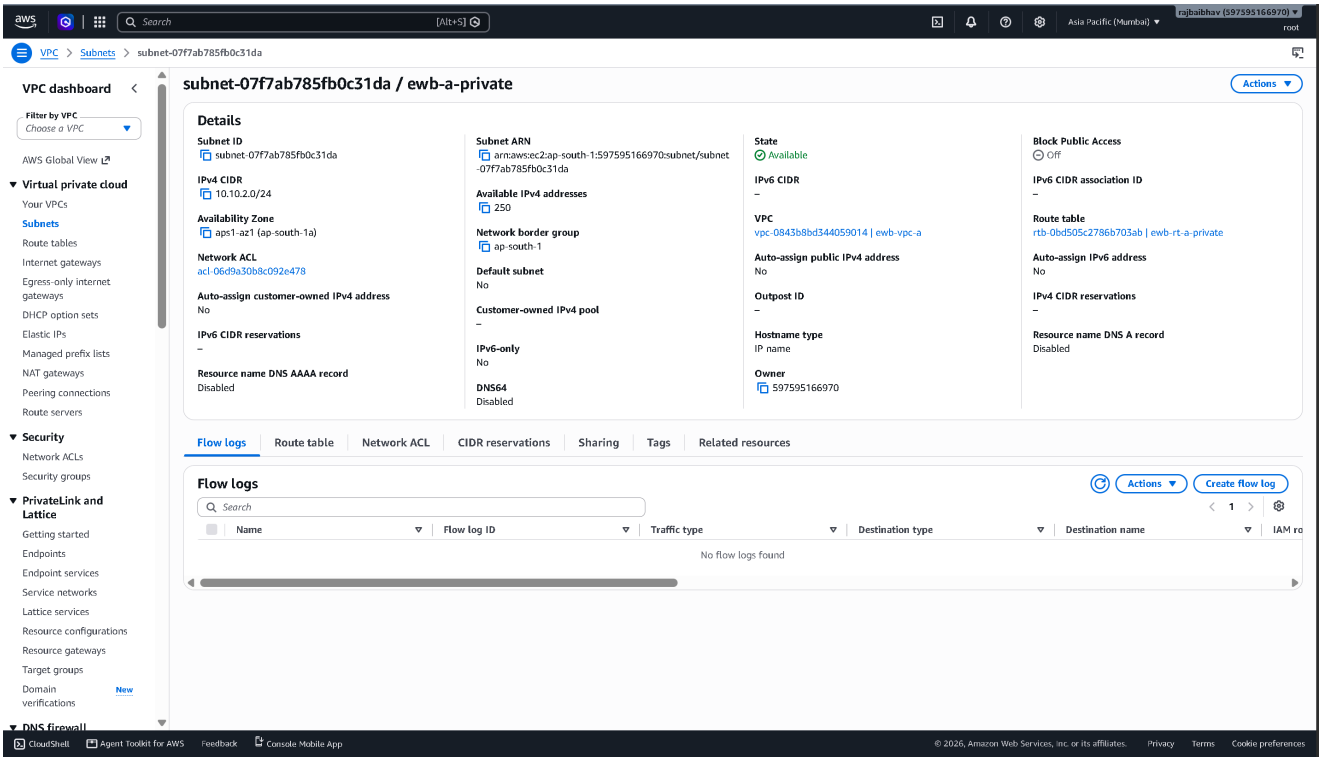
Checkpoint 01 - VPC B resource map | view: vpc-097437c5c5b04cb2a / ewb-vpc-b | test: is VPC B private-only | result: 1 private subnet, 0 network connections, no internet route anywhere.



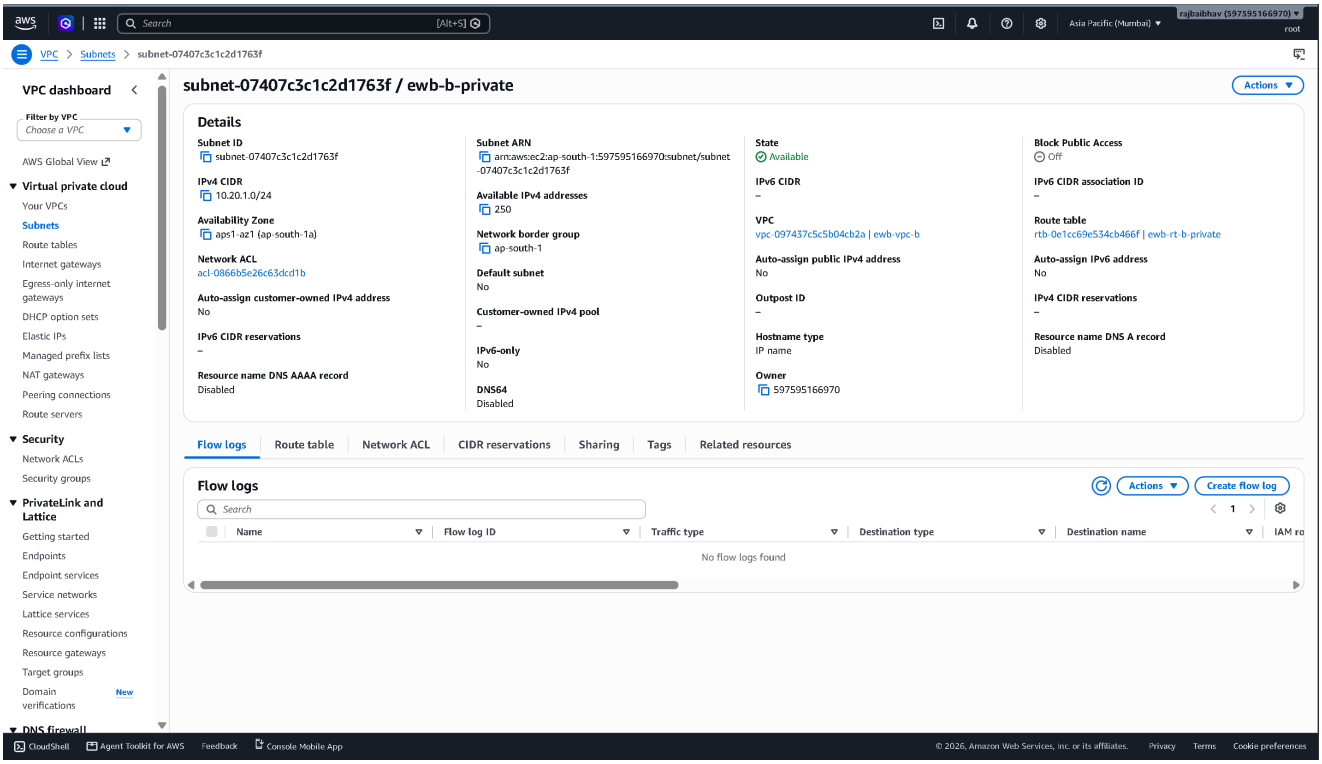
Checkpoint 01 - subnet association | view: subnet-0de70c3c480ad047c / ewb-a-public 10.10.1.0/24 | test: explicit association | result: uses custom ewb-rt-a-public, not the main table.



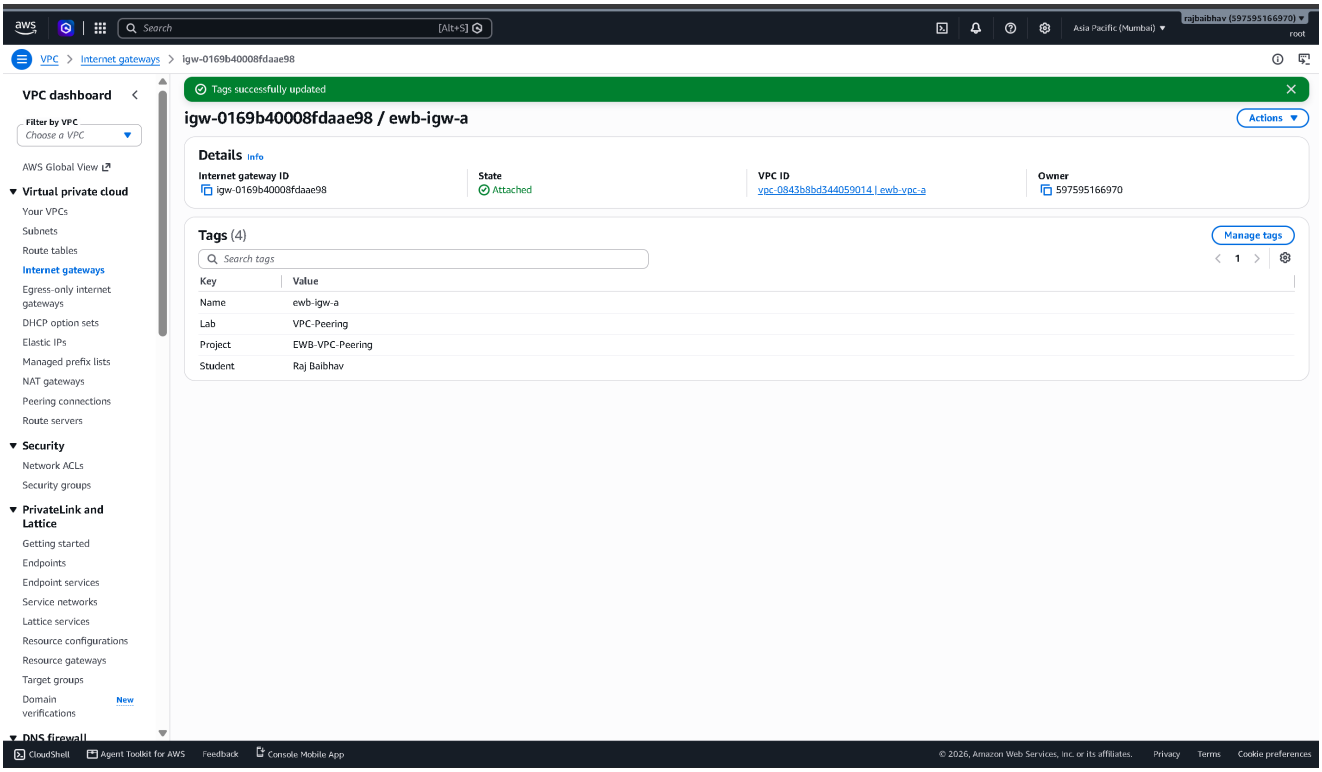
Checkpoint 01 - subnet association | view: subnet-07f7ab785fb0c31da / ewb-a-private 10.10.2.0/24 | test: explicit association | result: rtb-0bd505c2786b703ab / ewb-rt-a-private.



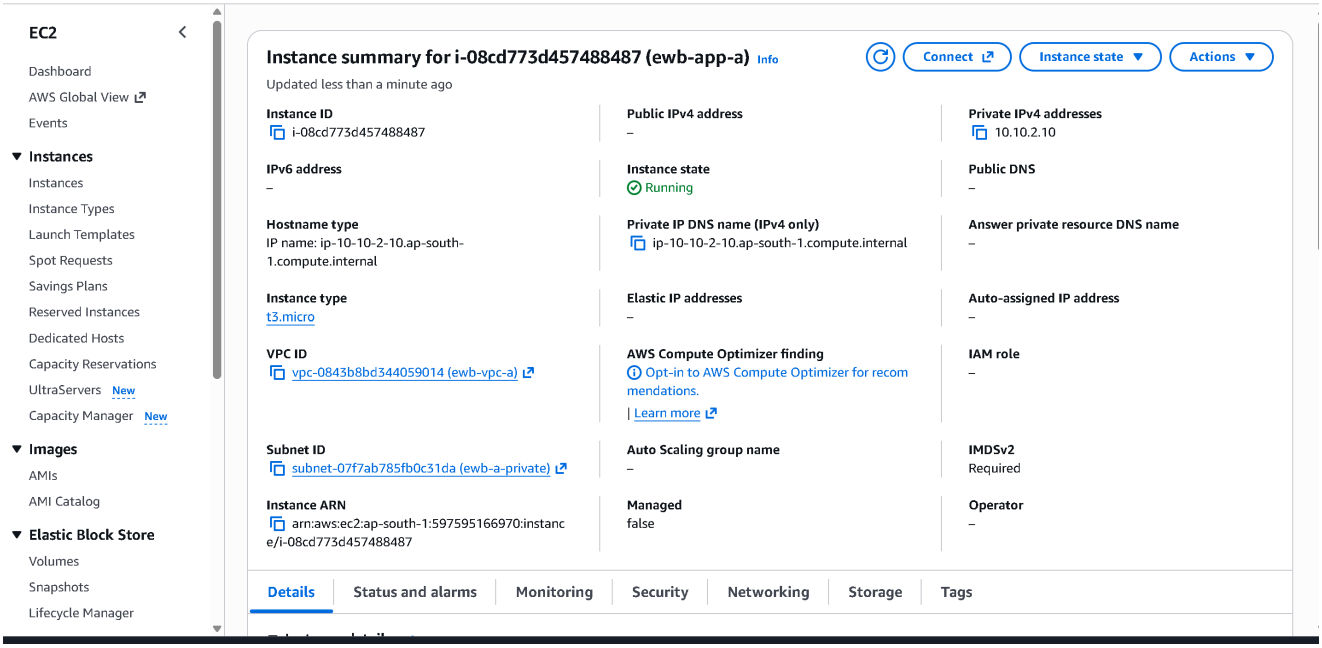
Checkpoint 01 - subnet association | view: subnet-07407c3c1c2d1763f / ewb-b-private 10.20.1.0/24 | test: explicit association | result: rtb-0e1cc69e534cb466f / ewb-rt-b-private.



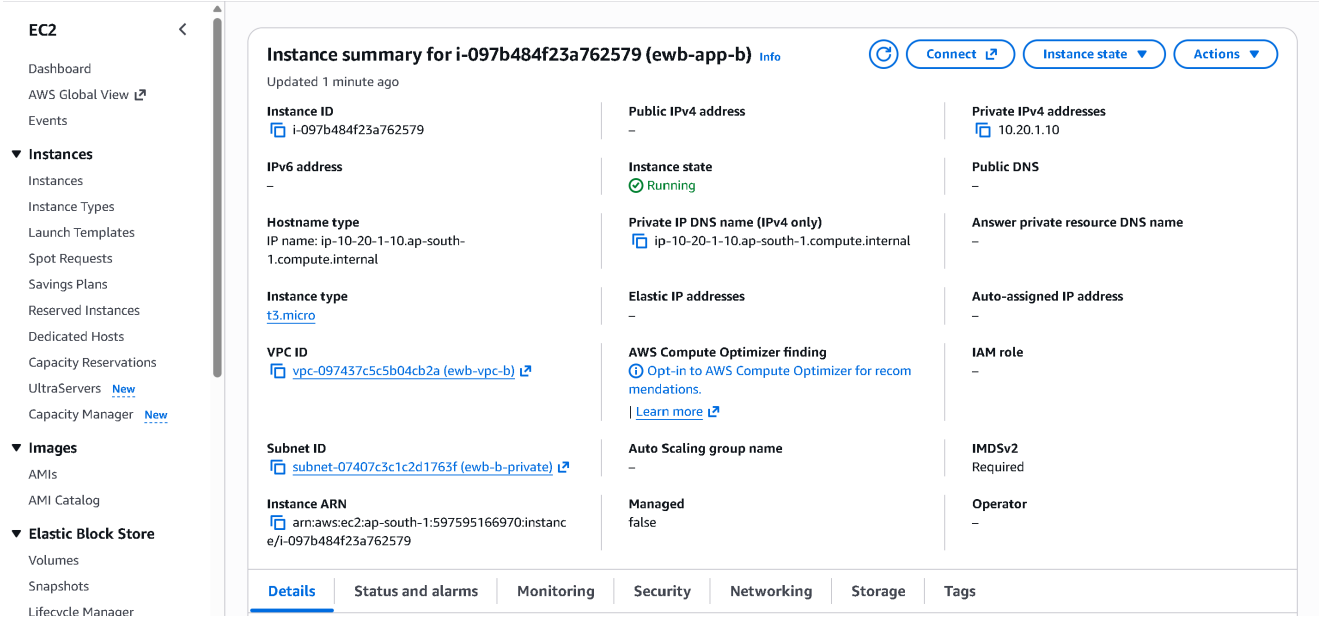
Checkpoint 01 - internet gateway | view: igw-0169b40008fdaae98 / ewb-igw-a | test: attachment scope | result: Attached to VPC A only, carrying the lab tags.



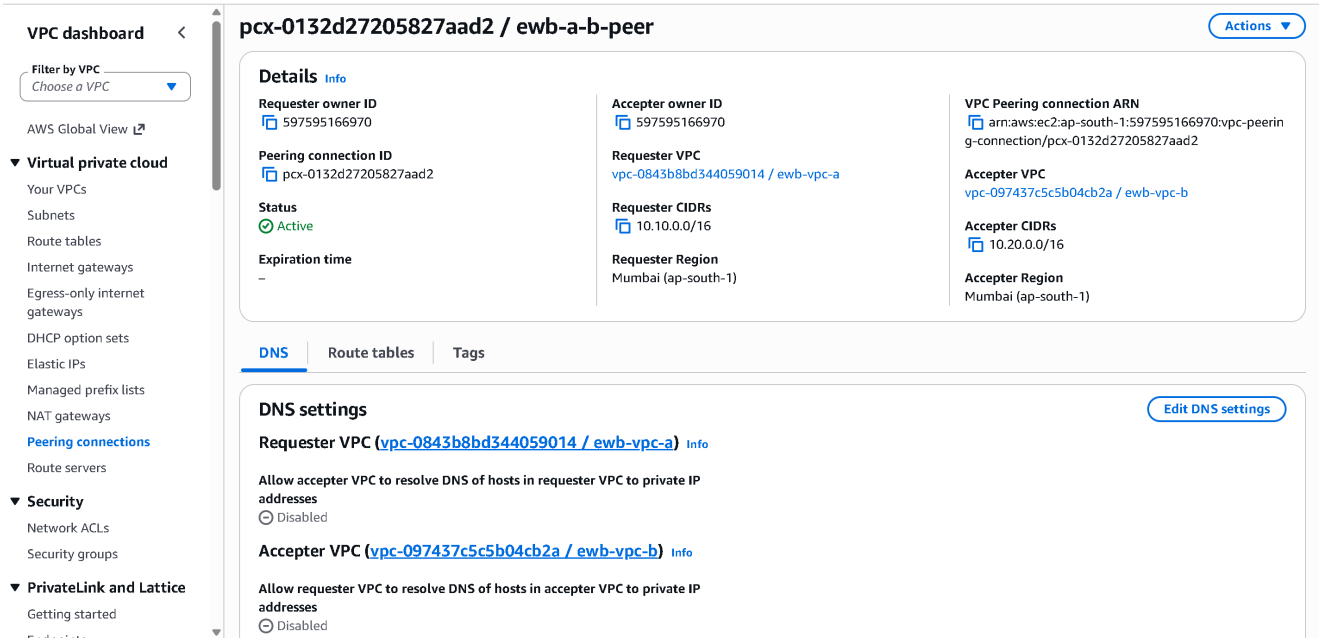
Checkpoint 02 - instance | host: ewb-app-a i-08cd773d457488487 | test: private-only host | result: private 10.10.2.10, Public IPv4 empty, IMDSv2 required.



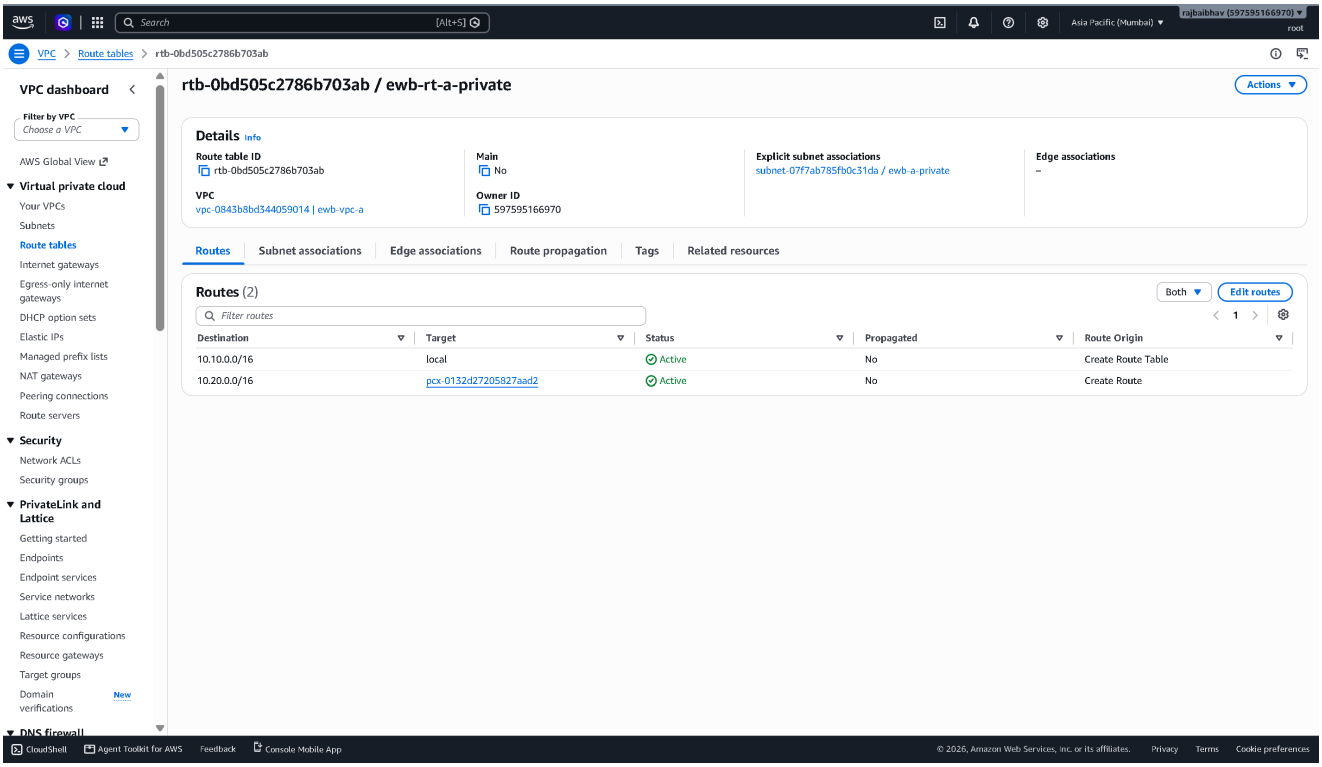
Checkpoint 02 - instance | host: ewb-app-b i-097b484f23a762579 | test: private-only host in the peered VPC | result: private 10.20.1.10, Public IPv4 empty. The bastion's own summary and its public IPv4 13.126.70.86 are in evidence/08.



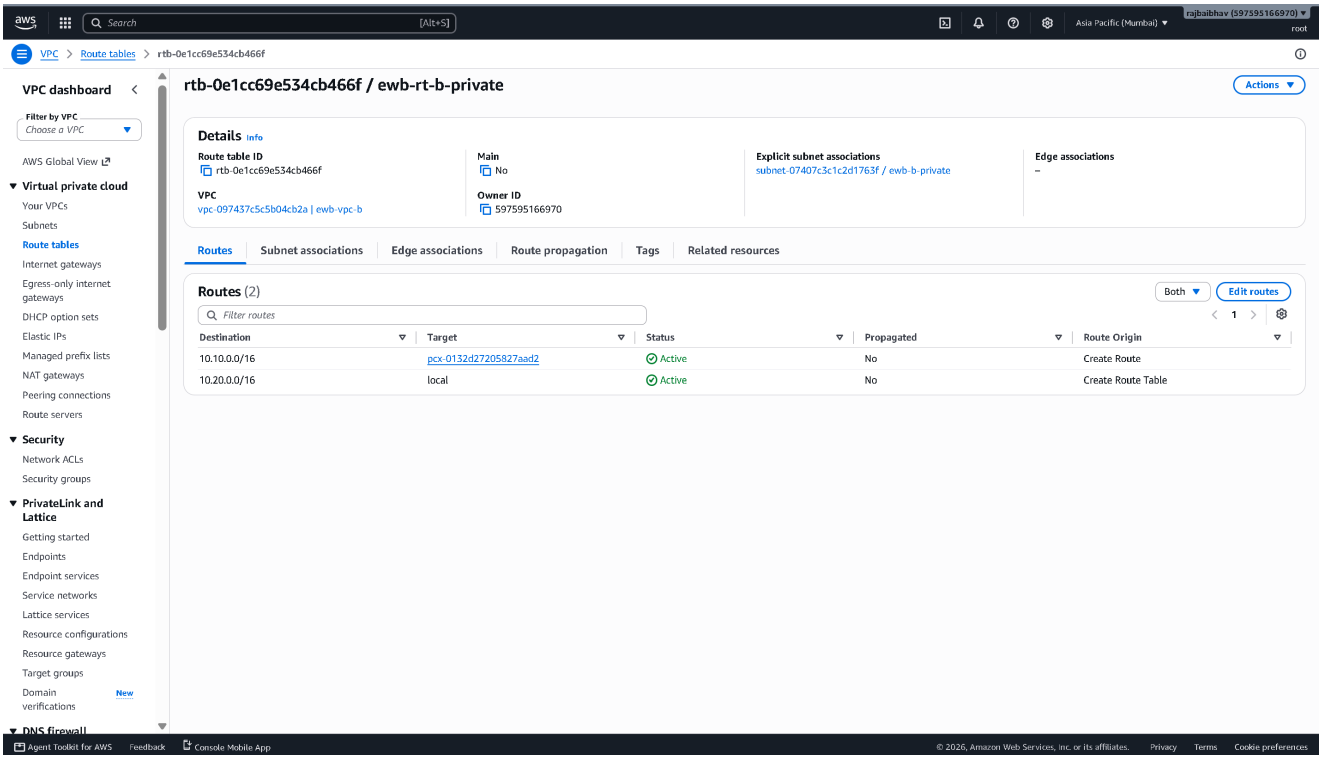
Checkpoint 03 - peering | test: is ewb-a-b-peer Active across both VPCs | result: pcx-0132d27205827aad2 Active, requester vpc-0843b8bd344059014 (10.10.0.0/16), accepter vpc-097437c5c5b04cb2a (10.20.0.0/16), both Mumbai.



Checkpoint 04 - routes | view: rtb-0bd505c2786b703ab / ewb-rt-a-private, associated with ewb-a-private | test: forward route with no default | result: local plus 10.20.0.0/16 to the pcx.



Checkpoint 04 - routes | view: rtb-0e1cc69e534cb466f / ewb-rt-b-private, associated with ewb-b-private | test: return route | result: 10.10.0.0/16 to the same pcx plus local. ewb-rt-a-public (0.0.0.0/0 to the IGW) is dumped in evidence/04.



6. Test results: baseline, forward, reverse, private SSH, isolation

Acceptance criteria met, in workbook order. Each block below is the real terminal transcript, not a summary of one.

#	Test	Host / command	Result
01	Baseline before peering	App-A: ping -c 4 10.20.1.10	0 received, 100% loss
02	Forward path	App-A: ping -c 4 10.20.1.10	4 received, 0% loss
03	Reverse path	App-B: ping -c 4 10.10.2.10	4 received, 0% loss
04	Private SSH	laptop -> bastion -> App-B	shell as ec2-user
05	Isolation	EC2 details, both app hosts	public=(none) on both

Test 01 - Baseline failure

```
# EWB VPC Peering evidence
# test      : 03-baseline-ping-app-a-to-app-b
# run at    : 2026-10-17T09:08:34Z
# from      : RAJBAIBHAV
# command   : ssh 10.10.2.10  ping -c 4 10.20.1.10

PING 10.20.1.10 (10.20.1.10) 56(84) bytes of data.

--- 10.20.1.10 ping statistics ---
4 packets transmitted, 0 received, 100% packet loss, time 3089ms

[ssh exit code: 1]
```

Host: ewb-app-a. Test: ping -c 4 10.20.1.10 before any peering exists. Result: 4 transmitted, 0 received, 100% loss. Expected, and the reference point for everything after. Transcript from the first build - the second attempt aborted on a stale host key, see section 7.

Test 02 - Forward path

```
# EWB VPC Peering evidence
# test      : 05-forward-ping-app-a-to-app-b
# run at    : 2026-10-17T12:18:04Z
# from      : RAJBAIBHAV
# command   : ssh 10.10.2.10  ping -c 4 10.20.1.10

PING 10.20.1.10 (10.20.1.10) 56(84) bytes of data.
64 bytes from 10.20.1.10: icmp_seq=1 ttl=127 time=0.502 ms
64 bytes from 10.20.1.10: icmp_seq=2 ttl=127 time=0.214 ms
64 bytes from 10.20.1.10: icmp_seq=3 ttl=127 time=0.213 ms
64 bytes from 10.20.1.10: icmp_seq=4 ttl=127 time=0.220 ms

--- 10.20.1.10 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3154ms
rtt min/avg/max/mdev = 0.213/0.287/0.502/0.124 ms

[ssh exit code: 0]
```

Host: ewb-app-a, after the peer routes. Result: 4 received, 0% loss. The request matched 10.20.0.0/16 in ewb-rt-a-private and crossed the peering connection.

Test 03 - Reverse path

```
# EWB VPC Peering evidence
# test      : 06-reverse-ping-app-b-to-app-a
# run at    : 2026-10-17T12:18:11Z
# from      : RAJBAIBHAV
# command   : ssh 10.20.1.10  ping -c 4 10.10.2.10

PING 10.10.2.10 (10.10.2.10) 56(84) bytes of data.
64 bytes from 10.10.2.10: icmp_seq=1 ttl=127 time=0.281 ms
64 bytes from 10.10.2.10: icmp_seq=2 ttl=127 time=0.226 ms
64 bytes from 10.10.2.10: icmp_seq=3 ttl=127 time=0.223 ms
64 bytes from 10.10.2.10: icmp_seq=4 ttl=127 time=0.295 ms

--- 10.10.2.10 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3126ms
rtt min/avg/max/mdev = 0.223/0.256/0.295/0.032 ms

[ssh exit code: 0]
```

Host: ewb-app-b. Result: 4 received, 0% loss. This only works because VPC B has the return route 10.10.0.0/16 pointing at the same peering connection.

Test 04 - Private SSH to App-B through the bastion

```
# EWB VPC Peering evidence
# test      : 07-ssh-app-b-via-bastion
# run at    : 2026-10-17T12:18:18Z
# from      : RAJBAIBHAV
# command   : ssh 10.20.1.10  hostname; hostname -I; whoami

ip-10-20-1-10.ap-south-1.compute.internal
10.20.1.10
ec2-user

[ssh exit code: 0]
```

Host: ewb-app-b, reached from the laptop with ProxyJump. Result: shell on ip-10-20-1-10 as ec2-user. No key was ever placed on the bastion.

Test 05 - Isolation and security groups

```
# test 05 - public IP check  2026-10-17T12:18:21Z
  ewb-app-a      i-08cd773d457488487    private=10.10.2.10    public=(none)  running
  ewb-bastion    i-0f9bbae63560aaff9    private=10.10.1.10    public=13.126.70.86  running
  ewb-app-b      i-097b484f23a762579    private=10.20.1.10    public=(none)  running

# security group inbound rules

### ewb-sg-bastion = sg-0a6467b762464b32f (inbound only)
  PORT         PROTOCOL     SOURCE
  22           tcp          47.15.154.141/32

### ewb-sg-app-a = sg-0b9037ae2a5b95caa (inbound only)
  PORT         PROTOCOL     SOURCE
  22           tcp          10.10.1.10/32
  type 8/code 0 icmp         10.20.1.10/32

### ewb-sg-app-b = sg-00fa26a69eb0b9db5 (inbound only)
  PORT         PROTOCOL     SOURCE
  22           tcp          10.10.1.10/32
  type 8/code 0 icmp         10.10.2.10/32
```

Result: App-A and App-B show public=(none). Only ewb-sg-bastion faces the internet, and every app rule is a /32 host source.

7. Controlled ICMP failure and recovery

The point of this test is to show which control actually carried the traffic. One inbound rule was deleted, nothing else changed, and the ping died even though the peering connection still read Active. Restoring the identical rule brought the replies back.

Test 06a - Controlled failure

```
# EWB VPC Peering evidence
# test      : 09-controlled-failure-ping
# run at    : 2026-10-17T09:17:25Z
# from      : RAJBAIBHAV
# command   : ssh 10.10.2.10  ping -c 4 10.20.1.10

PING 10.20.1.10 (10.20.1.10) 56(84) bytes of data.

--- 10.20.1.10 ping statistics ---
4 packets transmitted, 0 received, 100% packet loss, time 3125ms

[ssh exit code: 1]
```

Deleted only the inbound Echo request rule (type 8, code 0) from 10.10.2.10/32 on ewb-sg-app-b; the SSH rule stayed. Result: back to 100% loss while the peering connection still read Active. Transcript from the first build; same addresses, same rule, same outcome on the second.

Test 06b - Recovery

```
# EWB VPC Peering evidence
# test      : 10-recovered-ping
# run at    : 2026-10-17T09:17:51Z
# from      : RAJBAIBHAV
# command   : ssh 10.10.2.10  ping -c 4 10.20.1.10

PING 10.20.1.10 (10.20.1.10) 56(84) bytes of data.
64 bytes from 10.20.1.10: icmp_seq=1 ttl=127 time=0.324 ms
64 bytes from 10.20.1.10: icmp_seq=2 ttl=127 time=0.339 ms
64 bytes from 10.20.1.10: icmp_seq=3 ttl=127 time=0.323 ms
64 bytes from 10.20.1.10: icmp_seq=4 ttl=127 time=0.329 ms

--- 10.20.1.10 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3153ms
rtt min/avg/max/mdev = 0.323/0.328/0.339/0.006 ms
[ssh exit code: 0]
```

Restored that one identical rule. Result: 4 received again. This proves the security group, not routing, was the variable in step 06a.

8. Forward path and return path

- Forward: App-A 10.10.2.10 sends to 10.20.1.10. Its subnet table matches 10.20.0.0/16, target pcx-0132d27205827aad2. The packet crosses into VPC B and arrives at App-B's ENI, where the inbound Echo request rule lets it in.
- Return: App-B replies to 10.10.2.10. That is a fresh lookup in ewb-rt-b-private, which needs its own route 10.10.0.0/16 back over the same peering connection. Security groups are stateful, so the reply is accepted without a rule of its own.
- Remove either route and ping shows 100% loss while the connection still reads Active. A one-way route is the single easiest mistake to make in this lab.

9. Cleanup evidence

Teardown ran from the same script that built the lab, so it deleted only the IDs it had recorded. That matters here: the account also runs a production stack, and nothing outside the lab tags was touched.

```
# EWB VPC Peering - cleanup evidence
# run at  : 2026-10-17T10:13:26Z
# command : ./build-lab.sh teardown   (deletes only IDs recorded in state.env)

## lab VPCs, subnets, route tables, security groups, internet gateways, EBS volumes
## every one of these must be empty; the tag filter is Project=EWB-VPC-Peering
vpcs                 (empty = gone)
subnets              (empty = gone)
route-tables         (empty = gone)
security-groups      (empty = gone)
internet-gateways    (empty = gone)
ebs-volumes          (empty = gone)

## peering connection - final state
{
    "code": "deleted",
    "message": "Deleted by 597595166970"
}
   (AWS keeps a deleted pcx queryable for a short while; code=deleted is the end state)

## instances - all three reach terminated; AWS lists terminated instances for about an hour
i-03d1512b1a3659ff1     terminated
i-0f6a52081a6f8113b     terminated
i-05aa30d91ddf22920     terminated

## bastion public IPv4 13.206.123.132 was auto-assigned, not an Elastic IP, and released with the instance

## scope proof: the pre-existing production stack was not touched
vpc-065f1b55a9c404785   10.0.0.0/16

## left in place on purpose: EC2 key pair ewb-lab (no charge). Remove with:
##   aws ec2 delete-key-pair --region ap-south-1 --key-name ewb-lab
```

That was the first teardown, at 15:43 IST. The lab was built again for the console screenshots and torn down a second time; the check below ran after that, straight from the account.

```
# EWB VPC Peering - final account state after the second teardown
# run at  : 2026-10-17T12:58:09Z  (18:28 IST)
# command : read-only describe calls, filtered on Project=EWB-VPC-Peering

## every lab-tagged resource must be empty
  vpcs                 0
  subnets              0
  route-tables         0
  security-groups      0
  internet-gateways    0

## peering connections still listed by AWS (status must be deleted)
-------------------------------------
|    DescribeVpcPeeringConnections   |
+-------------------------+-----------+
|  pcx-0132d27205827aad2 |  deleted  |
+-------------------------+-----------+

## instances carrying the lab tag, any state (AWS keeps terminated rows for about an hour)
ewb-app-a   terminated
ewb-bastion terminated
ewb-app-b   terminated

## scope proof - VPCs left in the account (production and default only)
vpc-065f1b55a9c404785   10.0.0.0/16 smartgram-prod-vpc
vpc-058e02483e7d12187   172.31.0.0/16     None
```

10. What I learned

- Peering is only the link; route tables and security rules decide whether traffic may use it.
- Plan CIDRs before anything exists. 10.10.0.0/16 and 10.20.0.0/16 do not overlap, which is the reason peering was even allowed - separate /24 subnet ranges would not have fixed overlapping VPCs.
- Explicit subnet associations are worth naming. A route in the wrong table looks identical in a screenshot and does nothing.
- Test the negative case on purpose. The controlled failure taught me more than the passing ping.
- When a test fails and the console says healthy, capture packets at both ends instead of guessing which layer to blame.
- A rebuild is a new set of hosts even when the IPs match. Reusing private addresses means reusing SSH host keys that no longer belong to anything.
- Costs need an owner. t3.micro in three places plus one public IPv4 is cheap, but it is real money, so the script ends with teardown and a check that the account is clear.

11. The bugs that taught me the most

ICMP type 8, code 8

Peering was Active, all three route tables were correct, and SSH from the bastion to App-B worked - yet every ping failed in both directions. So I captured traffic on each side: tcpdump on App-A showed four echo requests leaving, and tcpdump on App-B showed zero packets arriving. The gap was in the hypervisor, before the guest ever saw the traffic.

The cause was my own security group rule. The AWS CLI shortcut --protocol icmp --port 8 wrote it as type 8, code 8. An Echo request is code 0, so nothing matched. In the console, picking 'Custom ICMP - IPv4 / Echo request' gives type 8 code 0 and would have been right from the start. The fix was an explicit --ip-permissions with FromPort=8 and ToPort=0. Two lessons: an Active peering connection proves nothing about traffic being permitted, and one packet capture per side tells you which side is lying where a screenshot would not.

Stale SSH host keys on the rebuild

Rebuilding the same lab a second time broke two tests before peering was even involved. The new instances came back with the same private IPs (10.10.2.10, 10.20.1.10) but fresh SSH host keys, so StrictHostKeyChecking refused the connection and the run died with exit 255. The transcript is kept as evidence/12-rebuild-hostkey-abort.txt because it is a real result, just not a lab result. The script now clears those IPs from secrets/known_hosts before the SSH phases, and run_local aborts the phase instead of quietly saving a host-key error as if it were a test.

The wider point: reusing addresses across builds is fine for the VPC design and dangerous for SSH trust. A rebuild is a new host, and it should be treated like one.

12. Viva answers

Why is the return route needed?
A ping is two trips. The reply to 10.10.2.10 is a new route lookup inside VPC B, which only knows its own CIDR until 10.10.0.0/16 points at the peering connection. Statefulness covers the security group, not the routing.

Why is App-B's SSH source the bastion's private IP?
The source of a packet is whoever sent it. ProxyJump makes the bastion open the connection from its primary private address 10.10.1.10. My laptop's public IP stops at the bastion and never crosses the peering link.

Why does Active peering not guarantee a successful ping?
The connection is only a link. Reachability still needs routes on both sides, the right inbound rule including the ICMP code, permissive NACLs, and running instances with the right destination IP.

Can VPC B use VPC A's internet gateway over peering?
No. An internet gateway serves the VPC it is attached to, and transitive routing through a peering connection is not supported. That is why dnf update fails on the private hosts, and why that failure is not a peering bug.

Core concept: peering is one-to-one and non-transitive. A-B plus A-C never creates B-C. Routes and rules stay editable after acceptance, and swapping either VPC means a new connection.

13. Reproduce it

GitHub: github.com/rajbaibhav1910/aws-vpc-peering-lab - one script, phased so each checkpoint can be captured in order. ./build-lab.sh all builds through the tests, failtest runs the controlled failure, teardown removes everything. Every phase finds its resources by Name tag before creating anything, so re-running is safe. The private key never left my laptop and is not in the repository.